Multivariate Linear Regression

General Principles

To study relationships between multiple continuous independent variables (e.g., the effect of weight and age on height), we can use a multiple regression approach. Essentially, we extend Linear Regression for continuous variable by adding a regression coefficient β_x for each continuous variable (e.g., β_{weight} and β_{age}).

Considerations

> **Note**
>
> - We have the same considerations as for the Regression for continuous variable.
> - The model interpretation of the regression coefficients β_x is considered for fixed values of the other independent variable(s)' regression coefficients—i.e., for a given age, β_{weight} represents the expected change in the dependent variable (height) for each one-unit increase in weight, holding all other variables (e.g., age) constant.

Example

Below is example code demonstrating Bayesian multiple linear regression using the BayesForge (BF) package. Data consist of three continuous variables (*height*, *weight*, *age*), and the goal is to estimate the effect of *weight* and *age* on *height*. This example is based on McElreath (2018).

Python

```python
from BayesForge import bf

# Setup device-------------------------------------------------
m = bf(platform='cpu')

# Import Data & Data Manipulation ------------------------------------------------
from importlib.resources import files
# Import
data_path = m.load.howell1(only_path = True)
m.data(data_path, sep=';')
m.df = m.df[m.df.age > 18] # Subset data to adults
m.scale(['weight', 'age']) # Normalize

# Define model -------------------------------------------------
def model(height, weight, age):
    # Parameter prior distributions
    alpha = m.dist.normal(178, 20, name = 'alpha')
    beta1 = m.dist.normal(0, 0.5, name = 'beta1')
    beta2 = m.dist.normal(0, 0.5, name = 'beta2')
    sigma = m.dist.uniform(0, 50, name = 'sigma')
    # Likelihood
    m.dist.normal(alpha + beta1 * weight + beta2 * age, sigma, obs = height)

# Run MCMC -----------------------------------------------
m.fit(model, progress_bar=False) # Optimize model parameters through MCMC sampling

# Summary -----------------------------------------------
m.summary()
```

```
An NVIDIA GPU may be present on this machine, but a CUDA-enabled jaxlib is not installed. Fal

bf v 0.0.60 package loaded
jax.local_device_count 16
```

	mean	sd	eti89_lb	eti89_ub	ess_bulk	ess_tail	r_hat	mcse_mean	mcse_sd
alpha	154.65	0.29	154.18	155.11	4534.89	3098.18	1.0	0.0	0.0
beta1	4.39	0.27	3.96	4.81	3820.20	3251.82	1.0	0.0	0.0
beta2	-0.03	0.25	-0.44	0.36	3901.39	3054.63	1.0	0.0	0.0
sigma	5.35	0.22	5.01	5.70	4038.36	3301.46	1.0	0.0	0.0

R

```r
library(BayesForge)
m=importBF(platform='cpu')

# Import Data & Data Manipulation ------------------------------------------------
m$data(m$load$howell1(only_path = T), sep=';')# Import
m$df = m$df[m$df$age > 18,] # Subset data to adults
m$scale(list('weight', 'age')) # Normalize
m$data_to_model(list('weight', 'height', 'age')) # Send to model (convert to jax array)

# Define model -------------------------------------------------
model <- function(height, weight, age){
  # Parameter prior distributions
  alpha = bf.dist.normal(178, 20, name = 'alpha')
  beta1 = bf.dist.normal(0, 0.5, name = 'beta1')
  beta2 = bf.dist.normal(0, 0.5, name = 'beta2')
  sigma = bf.dist.uniform(0, 50, name = 'sigma')
  # Likelihood
  bf.dist.normal(alpha + beta1 * weight + beta2 * age, sigma, obs=height)
}

# Run MCMC -----------------------------------------------
m$fit(model) # Optimize model parameters through MCMC sampling

# Summary ------------------------------------------------
m$summary() # Get posterior distributions
```

Julia

```julia
using PyBayesForge

# Setup device-------------------------------------------------
m = importBF(platform="cpu")

# Import Data & Data Manipulation ------------------------------------------------
# Import
data_path = m.load.howell1(only_path = true)
m.data(data_path, sep=';')
m.df = m.df[m.df.age > 18] # Subset data to adults
m.scale(["weight", "age"]) # Normalize
```

```
# Define model ----------------------------------------------
@BF function model(height, weight, age)
    # Parameter prior distributions
    alpha = m.dist.normal(178, 20, name = "alpha")
    beta1 = m.dist.normal(0, 0.5, name = "beta1")
    beta2 = m.dist.normal(0, 0.5, name = "beta2")
    sigma = m.dist.uniform(0, 50, name = "sigma")
    # Likelihood
    m.dist.normal(alpha + beta1 * weight + beta2 * age, sigma, obs = height)
end

# Run mcmc --------------------------------------------------
m.fit(model)  # Optimize model parameters through MCMC sampling

# Summary ---------------------------------------------------
m.summary() # Get posterior distributions
```

> **Caution**
>
> For R users, if you create the regression coefficient in a single call:
>
> ```
> betas = m$dist$normal(0, 0.5, name = 'regression_coefficients', shape = c(2))
> ```
>
> you need to index them starting by 0:
>
> ```
> m$dist$normal(alpha + betas[0] * weight + betas[1] * age, sigma, obs = height)
> ```

Mathematical Details

Frequentist formulation

We model the relationship between the independent variables $(X_{1i}, X_{2i}, ..., X_{[K,i]})$ and the dependent variable Y using the following equation:

$$Y_i = \alpha + \beta_1 X_{[1,i]} + \beta_2 X_{[2,i]} + ... + \beta_n X_{[K,i]} + \epsilon_i$$

Where:

- Y_i is the dependent variable for observation i.

- α is the intercept term.
- $X_{[1,i]}$, $X_{[2,i]}$, ..., $X_{[K,i]}$ are the values of the independent variables for observation i.
- β_1, β_2, ..., β_K are the regression coefficients.
- ϵ_i is the error term for observation i, and the vector of the error terms, ϵ, are assumed to be independent and identically distributed.

Bayesian formulation

In the Bayesian formulation, we define each parameter with priors . We can express the Bayesian model as follows:

$$Y_i \sim \text{Normal}(\alpha + \sum_{k=1}^{K} \beta_k X_{[K,i]}, \sigma)$$

$$\alpha \sim \text{Normal}(178, 20)$$

$$\beta_k \sim \text{Normal}(0, 0.5)$$

$$\sigma \sim \text{Uniform}(0, 50)$$

Where:

- Y_i is the dependent variable for observation i.
- α is the intercept term, with a Normal(178, 20) prior centred on a plausible adult height (height is not scaled).
- β_k are slope coefficients for the K distinct independent variables, with Normal(0, 0.5) priors (the predictors are scaled).
- $X_{[1,i]}$, $X_{[2,i]}$, ..., $X_{[K,i]}$ are the values of the independent variables for observation i.
- σ is a standard deviation parameter, which here has a Uniform prior that constrains it to be positive.

Model validation tests

- Comparison with Stan: `1b.Multiple continuous variables.py` fits the same model with BayesForge and Stan and compares the posterior distributions (KL divergence) and parameter recovery.
- Documentation examples: the Python, R and Julia examples of this page are run by the documentation tests (`Python/test_model_pages.py`, `R/test_r_model_pages.py`, `Julia/test_julia_model_pages.py`).

Reference(s)

McElreath, Richard. 2018. *Statistical Rethinking: A Bayesian course with examples in R and Stan*. Chapman; Hall/CRC.